

Базовый инструментарий Установка и управление сервером



Авторские права

© Postgres Professional, 2017–2024

Авторы: Егор Рогов, Павел Лузанов, Илья Баштанов, Алексей Береснев

Фото: Олег Бартунов (монастырь Пху и пик Бхрикути, Непал)

Использование материалов курса

Некоммерческое использование материалов курса (презентации, демонстрации) разрешается без ограничений. Коммерческое использование возможно только с письменного разрешения компании Postgres Professional. Запрещается внесение изменений в материалы курса.

Обратная связь

Отзывы, замечания и предложения направляйте по адресу:

edu@postgrespro.ru

Отказ от ответственности

Компания Postgres Professional не несет никакой ответственности за любые повреждения и убытки, включая потерю дохода, нанесенные прямым или косвенным, специальным или случайным использованием материалов курса. Компания Postgres Professional не предоставляет каких-либо гарантий на материалы курса. Материалы курса предоставляются на основе принципа «как есть» и компания Postgres Professional не обязана предоставлять сопровождение, поддержку, обновления, расширения и изменения.

Основные понятия

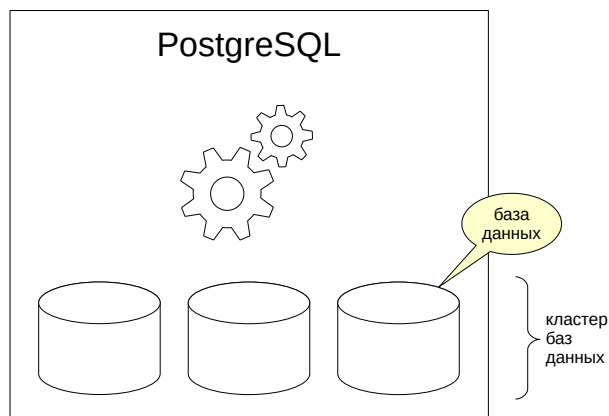
Установка из исходных кодов

Управление сервером

Пакетная установка

Управление сервером в Ubuntu

Облачные решения



Прежде чем говорить об установке, рассмотрим некоторые основные понятия.

PostgreSQL — программа, которая относится к классу *систем управления базами данных*.

Когда эта программа выполняется, мы называем ее *сервером PostgreSQL* или *экземпляром сервера*. Пока сервер представляется для нас «черным ящиком», но постепенно мы познакомимся с тем, как он устроен.

Данные, которыми управляет PostgreSQL, хранятся в *базах данных*. Один экземпляр PostgreSQL одновременно работает с несколькими базами данных. Этот набор баз данных называется *кластером баз данных*. Подробнее мы будем говорить о базах данных в теме «Организация данных. Базы данных и схемы».

Итак: кластер баз данных — это данные в файлах; сервер или экземпляр сервера — программа, управляющая кластером баз данных.

Установка из исходных кодов

- стабильная версия сервера
- можно использовать нестандартные параметры
- или собрать на специфичной архитектуре

Установка из репозитория git

- текущая версия сервера
- необходимо в первую очередь для разработчиков ядра;
- требует более широкого набора инструментов

Полезно иметь представление о том, как происходит установка из исходных кодов, чтобы в случае необходимости можно было собрать PostgreSQL с нестандартными параметрами или на специфичной архитектуре.

По адресу <https://www.postgresql.org/ftp/source> доступен исходный код в двух вариантах: gzip и bzip2. В виртуальной машине необходимый файл уже находится в домашнем каталоге пользователя student.

Исходные коды можно взять и непосредственно из git-репозитория проекта: `git://git.postgresql.org/git/postgresql.git` (проект имеет зеркала, в том числе на github).

В этом случае можно собрать не только стабильную версию, но и версию на любой конкретный коммит, в том числе наиболее актуальную.

При установке из репозитория git требуется более широкий набор инструментов. Например, лексический и синтаксический анализатор сделан с помощью Flex и Bison, и в git хранятся именно исходные коды для этих инструментов. При сборке из них генерируются файлы на Си, которые затем компилируются. При этом в архиве с исходными кодами находятся уже сгенерированные файлы Си.

Обязательно

tar, gzip/bzip2, GNU make, компилятор Си (C89)

Используются, но можно отключить

библиотеки GNU Readline, zlib, lz4, zstd

Дополнительно

языки программирования Perl, Python, Tcl
для использования PL/Perl, PL/Python, PL/Tcl

Kerberos, OpenSSL, OpenLDAP, PAM
для аутентификации и шифрования

библиотека ICU для кроссплатформенной поддержки UNICODE

Отдельные инструменты при сборке из репозитория git

5

Сборка и установка традиционно основывается на GNU Make и Autoconf:

<https://postgrespro.ru/docs/postgresql/16/install-make>

В этом курсе рассматривается традиционный способ сборки.

Начиная с версии 16 можно также использовать систему сборки программного обеспечения Meson:

<https://postgrespro.ru/docs/postgresql/16/install-meson>

Для сборки требуется ряд программ и утилит.

Библиотека readline дает возможность редактировать командную строку, пользоваться историей команд и автодополнением. В серверной инсталляции может быть и не нужна, если не предполагается запускать psql в консоли.

Библиотеки zlib, lz4 и zstd используются для сжатия.

Распаковка архива

Архив с исходными кодами находится в домашнем каталоге пользователя student. Распаковываем архив:

```
student$ cd
```

```
student$ tar xzf /home/student/postgresql-16.0.tar.gz
```

```
student$ ls -l /home/student/postgresql-16.0
```

```
total 876
-rw-r--r--  1 student student   365 сен 11  2023 aclocal.m4
drwxrwxr-x  2 student student  4096 сен 11  2023 config
-rwxr-xr-x  1 student student 584200 сен 11  2023 configure
-rw-r--r--  1 student student 87156 сен 11  2023 configure.ac
drwxrwxr-x 61 student student  4096 сен 11  2023 contrib
-rw-r--r--  1 student student  1192 сен 11  2023 COPYRIGHT
drwxrwxr-x  3 student student  4096 сен 11  2023 doc
-rw-r--r--  1 student student  4288 сен 11  2023 GNUmakefile.in
-rw-r--r--  1 student student   277 сен 11  2023 HISTORY
-rw-r--r--  1 student student 64592 сен 11  2023 INSTALL
-rw-r--r--  1 student student  1875 сен 11  2023 Makefile
-rw-r--r--  1 student student 101920 сен 11  2023 meson.build
-rw-r--r--  1 student student  6266 сен 11  2023 meson_options.txt
-rw-r--r--  1 student student  1213 сен 11  2023 README
drwxrwxr-x 16 student student  4096 сен 11  2023 src
```

Создание конфигурации

Настройку сборки выполним в каталоге с распакованным исходным кодом, используя скрипт configure.

```
student$ cd /home/student/postgresql-16.0
```

Внимание! Если сборка производится повторно, например с другими параметрами конфигурации, то предварительно нужно убрать файлы, оставшиеся от предыдущей сборки:

```
student$ make distclean
```

В команде configure можно указать различные параметры конфигурации. Например:

- `--prefix` — каталог установки, по умолчанию `/usr/local/pgsql`;
- `--enable-debug` — для включения отладочной информации.

Полный список параметров есть в документации.

Также принимаются во внимание переменные окружения. Например, `CC` и `CFLAGS` настраивают компилятор C.

Скрипт configure по умолчанию подготавливает сборку и установку исполняемых файлов, библиотек и прочих файлов PostgreSQL в каталог `/usr/local/pgsql`. Обычно каталог установки доступен для записи только суперпользователю ОС.

В целях демонстрации ключом `--prefix` укажем каталог установки ПО сервера в подкаталог `pgsql16` домашнего каталога пользователя student. Кроме того, будем использовать порт 5555 вместо порта 5432 по умолчанию:

```
student$ ./configure --prefix=/home/student/pgsql16 --with-pgport=5555
```

Вывод этой и следующих команд занимает очень много места, мы не будем его показывать.

Сборка и установка PostgreSQL

Возможные варианты:

- `make` — сборка только сервера;
- `make world` — сборка сервера, всех расширений и документации.

Собираем сервер. В зависимости от характеристик компьютера команда `make` может работать достаточно долго (минуты и даже десятки минут).

```
student$ make
```

Теперь устанавливаем сервер.

```
student$ make install
```

Для сборки расширений нужно повторить `make` и `make install` в подкаталоге `contrib`.

```
student$ cd contrib
```

```
student$ make
```

```
student$ make install
```

Если бы мы собирали сервер с расширениями и документацией (make world), то можно было бы установить все вместе командой make install-world.



Утилита

initdb

Особенности

новый кластер баз данных не может принадлежать суперпользователю ОС

PGDATA — удобная переменная для каталога кластера

файлы конфигурации создаются в каталоге PGDATA



желательно включить расчет контрольных сумм для страниц с данными

После установки сервера необходимо создать кластер баз данных. Для этого предназначена утилита `initdb`.

В целях безопасности каталог, в котором инициализируется кластер, не может принадлежать суперпользователю ОС. Владелец кластера обычно делают пользователя `postgres`.

Владелец кластера может определить переменную окружения `PGDATA`, указывающую на каталог кластера. Эту переменную используют некоторые утилиты сервера, когда им нужно узнать расположение кластера. К таким утилитам относится `initdb`, а также основная утилита управления сервером `pg_ctl`, о которой мы будем говорить дальше.

В процессе инициализации кластера утилита `initdb` создает конфигурационные файлы в этом же каталоге `PGDATA`. Подробнее о файлах конфигурации мы расскажем в одной из следующих тем.

Утилита `initdb` имеет множество ключей, влияющих на ее работу. Один из важных — `-k`, или `--data-checksums` — включает подсчет контрольных сумм на страницах данных. Проверка контрольной суммы будет выполняться при обращении к любой странице данных в кластере. Это несколько снижает производительность, но позволяет оперативно обнаружить повреждения в данных.

<https://postgrespro.ru/docs/postgresql/16/app-initdb>

Создание кластера

Теперь необходимо создать каталог для данных. Для промышленного использования владельцем каталога обычно назначают отдельного пользователя ОС. В нашем случае, пусть это будет каталог пользователя student внутри /home/student/pgsql16

```
student$ mkdir /home/student/pgsql16/data
```

Этот каталог часто называют PGDATA по имени переменной окружения, которую удобно использовать при работе с утилитами сервера.

```
student$ export PGDATA=/home/student/pgsql16/data
```

Все утилиты сервера установлены в подкаталоге bin, который стоит добавить в переменную PATH:

```
student$ export PATH=/home/student/pgsql16/bin:$PATH
```

Для инициализации кластера баз данных используется утилита initdb.

- В ключе -U можно указать имя суперпользователя СУБД. Если его не задать, то используется имя пользователя ОС, в нашем случае student. Обычно таким пользователем является postgres, так что зададим это имя явно.
- Ключ -k включает подсчет контрольных сумм страниц, что позволяет своевременно обнаруживать повреждения данных.
- Если переменная PGDATA не установлена, то в ключе -D следует указать каталог для данных. В нашем случае этот ключ можно было бы не указывать.

```
student$ initdb -U postgres -k -D /home/student/pgsql16/data
```



Утилита

`pg_ctl`

Основные задачи

запуск, останов и получение статуса сервера

обновление параметров конфигурации

переключение на реплику

К основным операциям управления сервером относятся запуск и останов сервера, получение текущего статуса сервера, обновление конфигурации и некоторые другие.

Для выполнения этих действий предназначена утилита `pg_ctl`, идущая в составе PostgreSQL.

Запускать `pg_ctl` следует от имени владельца кластера баз данных.

Более подробная информация об управлении сервером для администраторов баз данных:

<https://postgrespro.ru/docs/postgresql/16/app-pg-ctl>

<https://postgrespro.ru/docs/postgresql/16/runtime>

Управление сервером

Теперь все готово к запуску сервера.

- В ключе -l укажем файл журнала сообщений сервера.
- Ключ -D опускаем, поскольку задана переменная PGDATA.

```
student$ pg_ctl start -l /home/student/logfile
```

Для проверки работы можно вызвать утилиту psql, которая подключится к серверу и вернет текущее время:

```
student$ psql -U postgres -p 5555 -c 'SELECT now();'
```

```
      now
-----
2026-04-10 17:02:06.104584+03
(1 row)
```

Для останова сервера используется следующая команда:

```
student$ pg_ctl stop
```

Готовые пакеты — предпочтительный способ

Linux (**ubuntu**®, Debian, Red Hat, SUSE и другие)

входит в дистрибутив ОС

репозиторий (yum, apt) или пакеты RPM, DEB

FreeBSD, OpenBSD

пакеты из Ports and Packages Collection

Mac OS X

Windows

Предпочтительным вариантом является использование готовых пакетов, так как в этом случае получается понятная, поддерживаемая и легко обновляемая установка.

Пакеты существуют для большинства широко распространенных систем. В каждой из них могут быть свои особенности, с которыми стоит ознакомиться перед установкой.

В пакетные репозитории некоторых систем уже входит PostgreSQL, но, обычно, не самой последней версии. Актуальная версия может быть взята из репозитория PostgreSQL Global Development Group (PGDG):

<https://www.postgresql.org/download/>

Мы рассмотрим особенности пакета PostgreSQL для Ubuntu.



Утилита

pg_createcluster  initdb

Особенности

инициализация выполняется при установке пакета и создает кластер «main»



подсчет контрольных сумм по умолчанию не включен
расположение файлов конфигурации и журнала сервера
автоматический запуск сервера при старте ОС

Для инициализации кластера в Ubuntu создана обертка pg_createcluster над утилитой initdb.

Справку по использованию pg_createcluster можно получить командой:

```
$ man pg_createcluster
```

Утилита pg_createcluster автоматически запускается при установке пакета и создает кластер баз данных с именем «main».

Стоит обратить внимание, что инициализация кластера проходит с отключенным подсчетом контрольных сумм в страницах данных.

Исполняемые файлы, файлы конфигурации и журнал сервера размещены в соответствии с правилами, принятыми в Ubuntu.

Кроме того, настраивается автоматический запуск и останов сервера PostgreSQL при запуске и останове операционной системы.

Для удаления кластера используется утилита pg_dropcluster.

Утилиты pg_createcluster и pg_dropcluster специфичны для Ubuntu.

В других системах нужно явно инициализировать кластер с помощью initdb и задействовать подходящие средства операционной системы.

Создание кластера в Ubuntu

В виртуальной машине курса PostgreSQL установлен из пакета командой:

```
student$ sudo apt install -y postgresql-16
```

Каталог установки PostgreSQL:

```
student$ sudo ls -l /usr/lib/postgresql/16
```

```
total 8
drwxr-xr-x 2 root root 4096 ноя 25 17:25 bin
drwxr-xr-x 4 root root 4096 ноя 25 17:25 lib
```

Владелец ПО сервера — пользователь root.

Утилита pg_config показывает, с какими параметрами был собран сервер:

```
student$ sudo /usr/lib/postgresql/16/bin/pg_config --configure
```

```
'--build=x86_64-linux-gnu' '--prefix=/usr' '--includedir=${prefix}/include'
'--mandir=${prefix}/share/man' '--infodir=${prefix}/share/info' '--sysconfdir=/etc'
'--localstatedir=/var' '--disable-option-checking' '--disable-silent-rules'
'--libdir=${prefix}/lib/x86_64-linux-gnu' '--runstatedir=/run'
'--disable-maintainer-mode' '--disable-dependency-tracking' '--with-tcl' '--with-perl'
'--with-python' '--with-pam' '--with-openssl' '--with-libxml' '--with-libxslt'
'--mandir=/usr/share/postgresql/16/man' '--docdir=/usr/share/doc/postgresql-doc-16'
'--sysconfdir=/etc/postgresql-common' '--datarootdir=/usr/share/'
'--datadir=/usr/share/postgresql/16' '--bindir=/usr/lib/postgresql/16/bin'
'--libdir=/usr/lib/x86_64-linux-gnu/' '--libexecdir=/usr/lib/postgresql/'
'--includedir=/usr/include/postgresql/' '--with-extra-version= (Ubuntu
16.11-1.pgdg24.04+1)' '--enable-nls' '--enable-thread-safety' '--enable-debug'
'--disable-rpath' '--with-uuid=e2fs' '--with-gnu-ld' '--with-gssapi' '--with-ldap'
'--with-pgport=5432' '--with-system-tzdata=/usr/share/zoneinfo' 'AWK=mawk'
'MKDIR_P=/bin/mkdir -p' 'PROVE=/usr/bin/prove' 'PYTHON=/usr/bin/python3' 'TAR=/bin/tar'
'XSLTPROC=xsltproc --nonet' 'CFLAGS=-g -O2 -fno-omit-frame-pointer
-mno-omit-leaf-frame-pointer -flto=auto -ffat-lto-objects -fstack-protector-strong
-fstack-clash-protection -Wformat -Werror=format-security -fcf-protection
-fno-omit-frame-pointer' 'LDFLAGS=-Wl,-Bsymbolic-functions -flto=auto -ffat-lto-objects
-Wl,-z,relro -Wl,-z,now' '--enable-tap-tests' '--with-icu' '--with-llvm'
'LLVM_CONFIG=/usr/bin/llvm-config-19' 'CLANG=/usr/bin/clang-19' '--with-lz4'
'--with-zstd' '--with-systemd' '--with-selinux' '--enable-dtrace'
'build_alias=x86_64-linux-gnu' 'CPPFLAGS=-Wdate-time -D_FORTIFY_SOURCE=3' 'CXXFLAGS=-g
-O2 -fno-omit-frame-pointer -mno-omit-leaf-frame-pointer -flto=auto -ffat-lto-objects
-fstack-protector-strong -fstack-clash-protection -Wformat -Werror=format-security
-fcf-protection'
```

Кластер баз данных с именем main уже инициализирован и находится в каталоге /var/lib/postgresql/16/main.

Владельцем каталога является пользователь postgres. Вот содержимое каталога:

```
student$ sudo ls -l /var/lib/postgresql/16/main
```

```
total 92
drwx----- 1 postgres postgres 4096 ноя 25 18:06 base
drwx----- 1 postgres postgres 4096 апр 10 17:02 global
drwx----- 2 postgres postgres 4096 ноя 25 18:06 pg_commit_ts
drwx----- 2 postgres postgres 4096 ноя 25 18:06 pg_dynshmem
drwx----- 4 postgres postgres 4096 ноя 25 18:06 pg_logical
drwx----- 4 postgres postgres 4096 ноя 25 18:06 pg_multixact
drwx----- 2 postgres postgres 4096 ноя 25 18:06 pg_notify
drwx----- 2 postgres postgres 4096 ноя 25 18:06 pg_replslot
drwx----- 2 postgres postgres 4096 ноя 25 18:06 pg_serial
drwx----- 2 postgres postgres 4096 ноя 25 18:06 pg_snapshots
drwx----- 1 postgres postgres 4096 апр 10 17:01 pg_stat
drwx----- 2 postgres postgres 4096 ноя 25 18:06 pg_stat_tmp
drwx----- 2 postgres postgres 4096 ноя 25 18:06 pg_subtrans
drwx----- 2 postgres postgres 4096 ноя 25 18:06 pg_tblspc
drwx----- 2 postgres postgres 4096 ноя 25 18:06 pg_twophase
-rw----- 1 postgres postgres   3 ноя 25 18:06 PG_VERSION
drwx----- 3 postgres postgres 4096 ноя 25 18:06 pg_wal
drwx----- 2 postgres postgres 4096 ноя 25 18:06 pg_xact
-rw----- 1 postgres postgres  88 ноя 25 18:06 postgresql.auto.conf
-rw----- 1 postgres postgres 130 апр 10 17:01 postmaster.opts
-rw----- 1 postgres postgres 109 апр 10 17:01 postmaster.pid
```




Утилита

pg_ctlcluster

 pg_ctl

Основные задачи

- запуск, останов и получение статуса сервера
- перечитывание параметров конфигурации
- переключение на реплику

В пакетном дистрибутиве для Ubuntu доступ к утилите `pg_ctl` осуществляется не напрямую, а через специальную обертку `pg_ctlcluster`. Справку по использованию `pg_ctlcluster` можно получить командой:

```
$ man pg_ctlcluster
```

В других системах утилита `pg_ctl` может использоваться непосредственно.

Управление сервером в Ubuntu

При установке из пакета в настройки запуска ОС добавляется запуск PostgreSQL. Поэтому после загрузки операционной системы отдельно стартовать PostgreSQL не нужно.

Можно явным образом управлять сервером с помощью утилиты `pg_ctlcluster`, которая запускается либо от имени пользователя ОС `postgres`, либо через `sudo`.

Утилита `pg_ctlcluster` позволяет управлять несколькими серверами разных версий. В качестве параметров ей передаются:

- 16 — номер версии сервера;
- `main` — название сервера;
- команда

Остановить сервер:

```
student$ sudo pg_ctlcluster 16 main stop
```

```
student$ pg_lsclusters 16 main
```

Ver	Cluster	Port	Status	Owner	Data directory	Log file
16	main	5432	down	postgres	/var/lib/postgresql/16/main	/var/log/postgresql/postgresql-16-main.log

Запустить сервер:

```
student$ sudo pg_ctlcluster 16 main start
```

Перезапустить:

```
student$ sudo pg_ctlcluster 16 main restart
```

Получить текущий статус сервера:

```
student$ sudo pg_ctlcluster 16 main status
```

```
pg_ctl: server is running (PID: 58989)
/usr/lib/postgresql/16/bin/postgres "-D" "/var/lib/postgresql/16/main" "-c"
"config_file=/etc/postgresql/16/main/postgresql.conf"
```

Перечитать файлы конфигурации:

```
student$ sudo pg_ctlcluster 16 main reload
```

Журнал сообщений сервера при пакетной установке находится здесь:

```
student$ ls -l /var/log/postgresql/postgresql-16-main.log
```

```
-rw-r----- 1 postgres adm 46002 apr 10 17:02 /var/log/postgresql/postgresql-16-main.log
```

Заглянем в конец журнала:

```
student$ tail -n 10 /var/log/postgresql/postgresql-16-main.log
```

```
2026-04-10 17:02:10.646 MSK [58935] LOG:  shutting down
2026-04-10 17:02:10.650 MSK [58935] LOG:  checkpoint starting: shutdown immediate
2026-04-10 17:02:10.676 MSK [58935] LOG:  checkpoint complete: wrote 3 buffers (0.0%); 0
WAL file(s) added, 0 removed, 0 recycled; write=0.008 s, sync=0.005 s, total=0.030 s;
sync files=2, longest=0.003 s, average=0.003 s; distance=0 kB, estimate=0 kB;
lsn=0/1922118, redo lsn=0/1922118
2026-04-10 17:02:10.683 MSK [58934] LOG:  database system is shut down
2026-04-10 17:02:10.891 MSK [58989] LOG:  starting PostgreSQL 16.11 (Ubuntu
16.11-1.pgdg24.04+1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu
13.3.0-6ubuntu2~24.04) 13.3.0, 64-bit
2026-04-10 17:02:10.891 MSK [58989] LOG:  listening on IPv4 address "127.0.0.1", port 5432
2026-04-10 17:02:10.894 MSK [58989] LOG:  listening on Unix socket
"/var/run/postgresql/.s.PGSQL.5432"
2026-04-10 17:02:10.906 MSK [58992] LOG:  database system was shut down at 2026-04-10
17:02:10 MSK
2026-04-10 17:02:10.917 MSK [58989] LOG:  database system is ready to accept connections
2026-04-10 17:02:13.344 MSK [58989] LOG:  received SIGHUP, reloading configuration files
```

PostgreSQL в виртуальной среде

- накладные расходы на виртуализацию
- администратор имеет полный доступ к экземпляру

PostgreSQL as a Service

- предлагается многими облачными провайдерами
- основное администрирование берет на себя провайдер
- доступ только к ограниченным инструментам для управления, резервного копирования, мониторинга

Средства виртуализации могут использоваться и для баз данных, в том числе PostgreSQL. Надо понимать, что за удобства этой технологии приходится расплачиваться накладными расходами. Для высоконагруженных систем любые дополнительные промежуточные слои между СУБД и железом нежелательны.

Кроме того, многие ведущие зарубежные и российские облачные провайдеры предлагают использовать PostgreSQL как сервис (Database as a Service, managed database).

В этом случае провайдер берет на себя большинство задач администрирования. Полного доступа к экземпляру в этом случае нет, а возможности управления ограничиваются предоставленными инструментами для таких задач, как мониторинг производительности или резервное копирование и восстановление.

В рамках курса мы не будем рассматривать особенности отдельных облачных решений. Для этого стоит изучить документацию, предоставляемую провайдерами услуг.

Два варианта установки — пакеты или исходный код

Доступны облачные решения

Сервер выполняется под выделенным пользователем ОС

Перед использованием сервера необходимо
инициализировать кластер баз данных

Специальные команды для управления сервером

Включение расчета контрольных сумм в кластере.

1. Остановите сервер.
2. Проверьте, рассчитываются ли контрольные суммы в кластере.
3. Включите расчет контрольных сумм.
4. Запустите сервер.

2, 3. Используйте утилиту `pg_checksums`. Для ее запуска укажите полный путь: `/usr/lib/postgresql/16/bin/pg_checksums`

<https://postgrespro.ru/docs/postgresql/16/app-pgchecksums>

1. Останов сервера

```
student$ sudo pg_ctlcluster 16 main stop
```

2. Проверка

Чтобы узнать, включен ли расчет контрольных сумм страниц, запустим утилиту `pg_checksums` с ключом `--check`:

```
student$ sudo /usr/lib/postgresql/16/bin/pg_checksums --check -D /var/lib/postgresql/16/main  
pg_checksums: error: data checksums are not enabled in cluster
```

3. Включение расчета контрольных сумм

Выполняем `pg_checksums` с ключом `--enable`:

```
student$ sudo /usr/lib/postgresql/16/bin/pg_checksums --enable -D /var/lib/postgresql/16/main  
  
Checksum operation completed  
Files scanned: 1244  
Blocks scanned: 3749  
Files written: 1026  
Blocks written: 3749  
pg_checksums: syncing data directory  
pg_checksums: updating control file  
Checksums enabled in cluster
```

Расчет контрольных сумм включен.

4. Запуск сервера

```
student$ sudo pg_ctlcluster 16 main start
```

1. Установите PostgreSQL из исходных кодов так, как это показано в демонстрации.
Создайте кластер баз данных, запустите сервер.
Убедитесь, что сервер работает.
Остановите сервер.

1. Сборка PostgreSQL из исходных кодов

Все необходимые команды были показаны в демонстрации, но они не выполнялись. Повторите эти действия своими руками в виртуальной машине курса.

Имейте в виду, что компиляция исходных кодов может занимать продолжительное время (от нескольких минут до десятков минут в зависимости от мощности компьютера).